

Hierarchical Multi-Label Text Classification

NOAM CATHERINE, Korea University Exchange Student (2025952809)

1 Reference

This study is inspired by the work presented in [NAACL 2021 paper on hierarchical classification](#).

2 Experimental Setup

GPU: NVIDIA RTX 3060 ||| RAM: 32 GB

3 Introduction

Hierarchical multi-label text classification is a challenging task in natural language processing (NLP), particularly when performed without access to labeled data. In contrast to traditional supervised settings, where classification models rely on annotated corpus, this project requires building a system capable of assigning each product review one or more categories drawn from a taxonomy of 531 classes, while only using unlabeled data. The categories are organized in a directed acyclic graph, making the classification problem substantially more complex due to both hierarchical dependencies and the multi-label nature of the task (see Annexe 1).

The core challenge of this project lies in developing an approach that can leverage the available resources (a corpus of 29,487 unlabeled train reviews, a corpus of 19,658 unlabeled train reviews, a hierarchical class taxonomy, and class-specific keywords) to generate sufficiently reliable silver labels that can serve as training signals. This situates the problem within the broader context of weak supervision and self-supervised learning, where the goal is to approximate high-quality annotations without manual labeling.

Beyond generating pseudo-labels, the task also requires designing a robust classifier capable of learning from noisy supervision while exploiting the structure of the taxonomy. This may involve pretrained language models, self-training mechanisms, or graph-based architectures that explicitly model parent-child relationships among classes.

This report outlines the methodology used to address the task. I begin by describing the strategy for generating silver labels and the heuristics behind it. I then summarize the training process, including the learning objectives and techniques used to improve model performance. Next, I present the model architecture and prediction pipeline. Then the experimental section compares several approaches, such as self-training and GNN-based methods, and evaluates their results. Finally, I include a brief case study highlighting one successful prediction and one failure, illustrating the model's strengths and limitations.

4 Silver Label Generation

For me, this stage is the most critical. In self-supervised learning, everything hinges on generating labels that are as close to the true distribution as possible. Even the most powerful classifier will fail if the supervision signal is too noisy, since the model ends up learning noise rather than meaningful patterns.

As mentioned earlier, the primary objective is to generate labels that best reflect both the true underlying semantics and the hierarchical structure of the taxonomy. The first step is therefore to adopt a word-level semantic approach. I initially experimented with BM25, but it quickly proved

Author's Contact Information: Noam CATHERINE, Korea University Exchange Student (2025952809).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2025/12-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

insufficient: it ignores context, cannot handle synonyms and lacks any real semantic understanding (lexical understanding). For this reason, I moved fastly to an embedding-based similarity method, computing semantic similarity between the corpus and the class labels using pretrained sentence transformers models.

Using such models for the corpus is straightforward, since the input consists of full review sentences. The situation is less obvious for the class labels, which are often only one or two words (not good for sentence transformers). To remedy this, I first preprocess each class name by removing elements that may confuse the model (such as underscores). Then, I enrich these labels using the keywords provided in the class related file (adding semantic context => data augmentation) as well as the keywords associated with their parent category (adding hierarchical context). This transformation turns each class into a short but meaningful collection of semantically relevant terms. In this form, the labels become perfectly suited for sentence-transformer models, which perform extremely well when given groups of words containing enough contextual and hierarchical information to produce robust embeddings.

Once the enriched representations are constructed, obtaining the embedding vectors for both reviews and classes become straightforward, allowing us to compute similarity scores between them. For this step, I rely on cosine similarity rather than an inner product: since the absolute magnitude of the embeddings is not meaningful in this context. This approach enables us to identify what can be considered the core candidate classes for each review ((see Annexe 2).

However, these core classes can still originate from completely different branches of the taxonomy. In addition, by examining the format of the 5 gold labels provided by Kaggle, I observe a consistent pattern: the labels typically correspond to a core class, its parent, and its grandparent when applicable. Based on this observation, I adopted the heuristic that predictions should follow this format of hierarchical structure. Thus, instead of selecting the top 3 (or more) classes by similarity (unnecessary noise) I opted for a stricter approach: only the top 1 core class is selected, even when the top 2 score is close.

I experimented two methods of generation. The first selects only the top core class and serves as a simple baseline. The second better captures the hierarchy: it starts with the top class and, using embeddings adapted to reflect the DAG structure, expands it to include parent and ancestor categories when applicable. This produces a more consistent and structured label set. Indeed, by integrating structural information directly into the embeddings, this hierarchy-aware representation becomes better suited for guiding the expansion step, whereas the semantic embedding space remains dedicated to identifying the core class itself. This separation of roles helps provide label reliability by reducing inconsistencies and ensuring that the final silver labels align both semantically and hierarchically.

After establishing this clean baseline, I explored ways to further improve the predictions. The first avenue was simply the choice of sentence-transformer model. I initially followed the recommendation from the original paper and experimented with a RoBERTa-based model, namely sentence-transformers/all-distilroberta-v1.

However, upon reviewing the sentence-transformer model leaderboard (https://www.sbert.net/docs/sentence_transformer/pretrained_models.html), I found that more performant architectures were available for English texts. Among them, all-mpnet-base-v2 stood out as a strong candidate and ultimately became my best-performing model (see Annexe 3).

For your information, it was not fine-tuned on Amazon product data ; it was trained on large and diverse corpora such as Reddit, Yahoo Answers, and similar sources (e.g., <https://huggingface.co/optimum/all-MiniLM-L6-v2>), which provided excellent generalization. In practice, this model yielded good label coverage (97%) and high similarity to the 5 gold labels provided by Kaggle, making it a particularly appealing choice for the task.

I also experimented with a method similar to the one used in assignment 3, combining the semantic similarity from my pretrained embedding model with the similarity score of a purely lexical model (BM25). However, although lexical models such as BM25 provide a useful complementary signal, they ultimately bring only marginal improvements in this setting. This confirmed that

relying primarily on semantic sentence-transformer models is the most effective approach for this task.

Finally, to further improve the quality of my silver labels, I finally adopted an ensemble strategy. I combined the predictions of three different embedding-based models (mpnet, Roberta, miniLM) plus a BM25 system coupled to sentence transformers, and I retained only the labels that appeared in most of the models (vote by majority). This approach filters out uncertain labels while preserving hierarchical consistency, since each individual model produces a coherent hierarchy. As a result, removing non-consensual labels still leaves either a valid hierarchy, a single label, or nothing, but never a broken structure. This is how I constructed my most reliable silver labels, achieving a 96.8 percent coverage while preserving hierarchical structure (for the case of multi-labels silver labels). Additionally, I created what I call pseudo-gold labels: these are the labels for which all five models agree. They represent the cases where the model ensemble exhibits near-certain confidence; they will serve for self training classification method.

N.B : My results seems coherent. For example, my first silver multi labels are “systems accessories”, “medical supplies equipment” and “health personal care” for the text : “0 omron hem 790it automatic blood pressure monitor with advanced omron health management software so far this machine has worked well and is very simple to use . it is nice to have immediate feedback on the bloodpressure effects of my various exercises , food consumption , and relaxation or stress levels”.

5 Training process

5.1 Training Data

For training, I use both the training and test corpus silver labels to maximize generalization, as permitted by the instructions: “You are allowed to use the test corpus information during training.” This provides broader lexical and semantic coverage for the model.

5.2 Loss Functions

The main model is trained using a Binary Cross-Entropy loss, defined as

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C \left[y_{i,c} \log \sigma(\hat{y}_{i,c}) + (1 - y_{i,c}) \log(1 - \sigma(\hat{y}_{i,c})) \right],$$

where N is the number of samples, C the number of classes, $y_{i,c} \in \{0, 1\}$ the true label, $\hat{y}_{i,c}$ the predicted logit, and σ the sigmoid function. This loss is standard for multi-label classification, encouraging independent label activation while allowing multiple classes per sample. Starting from this baseline, I experimented with additional components (soft BCE, smoothed BCE, etc.), but tuning their weights proved time-consuming and often yielded only marginal improvements.

I therefore opted for a simple yet effective extension of the base loss, built upon a weighted Binary Cross-Entropy objective (BCEWithLogitsLoss with a positive class weight of 20), which helps address class imbalance while preserving the simplicity and stability of the baseline formulation. Moreover, the BCE term is augmented with a hierarchical penalty that enforces consistency between parent and child probabilities (penalty if the condition is not respected : $P(\text{parent}) > P(\text{children})$), discouraging situations where a child node is predicted with high confidence while its parent remains unlikely. This directly exploits the known structure of the taxonomy and encourages hierarchically coherent predictions.

To regularize self-training, I add a consistency loss on unlabeled data: small perturbations (Gaussian noise) are applied to inputs and the model is encouraged to keep predictions and embeddings close to those of a fixed teacher, yielding a total loss of $\text{total_loss} = \text{supervised_loss} + \lambda_{\text{cons}} \cdot \text{consistency_loss}$, which stabilizes training and reduces overfitting to noisy silver labels.

Finally, For comparison, I also experimented with the taxonomy loss proposed in the main paper (using only 1 core class). This loss combines a Binary Cross-Entropy formulation with hierarchical constraints, separately penalizing positive and negative labels. Denoting $p \in [0, 1]^C$ the predicted probabilities for C classes, and using masks $m_i^+, m_i^- \in \{0, 1\}^C$ for positives (core class and its parents) and negatives (all other classes excluding the children of the core class), the loss for a single sample

is:

$$\mathcal{L}_{\text{taxonomy}} = - \sum_{c=1}^C \left[m_c^+ \log p_c + m_c^- \log(1 - p_c) \right].$$

This encourages high probabilities along valid parent–child paths while suppressing incorrect activations, yielding structured predictions that respect the hierarchy.

5.3 Regularization methods

Because the silver labels are already very strong and relatively rigid, there is a significant risk that the classifier simply memorizes them instead of learning more generalizable patterns. As mentioned in the silver label generation section, these labels already account for a large portion of the final performance (around 70–80 percent in practice), which makes it crucial to regularize the training process so that the model can go beyond merely copying them.

To improve generalization, I employ standard regularization techniques during training. These include dropout in the classification head, use of weight decay, early stopping and experiments with different learning rates. In addition, I explored label masking, where part of the target vector is temporarily hidden so that the model learns to reconstruct missing labels based on the remaining ones. All these strategies share the same objective: to prevent overfitting to potentially noisy silver labels and to encourage the classifier to rely on robust patterns in the embedding space rather than memorizing specific label configurations.

Finally, I explored more advanced approaches inspired by the second assignment, where the goal is to encourage the model to “free itself” from the initial silver labels through self-training on different models. This pseudo-labeling (add hierarchical + remove each n epochs) procedure is complemented by temporal ensembling and consistency regularization (via mse loss on predictions and embeddings + Teacher/student ensemble), which stabilize the predictions and reduce the noise inherent in pseudo-labels.

Finally, for self-training, I combined the pseudo-gold labels with the predictions from my best model (linear model). For each document, if at least one label appeared in both sets, I retained all labels from both sources for that document, ensuring a more comprehensive set of high-confidence annotations.

6 Models and Prediction Method

6.1 Linear (see Annexe 4)

In this setup, predictions are produced using a linear classifier applied on top of fixed sentence embeddings. The entire corpus (train and test) is first encoded using a pretrained model (e.g : Mpnet), generating dense vector representations for each sample. These embeddings serve as the input features for a simple linear layer that directly maps them to the label space. Formally, for each encoded sample x , the prediction is obtained as: $y = Wx + b$ where w and b are the parameters of the linear classifier. The model is trained in a fully supervised manner on the MPNet embeddings of the training split, with no fine-tuning of the encoder and no additional nonlinear components. Because the encoder remains frozen and the prediction head is linear, this method provides a strong and efficient baseline by evaluating how linearly separable the MPNet embedding space is for the task.

6.2 InnerProduct

I will not elaborate much on this model, as it is essentially based on Assignment 2. It relies on MPNet-based embeddings and performs classification by computing similarity scores (inner products) between review representations and label embeddings, capturing semantic alignment between inputs and labels.

6.3 GCN

To improve the original inner-product classifier, I extend it with a graph-based refinement of the label representations. The overall architecture now functions as a dual-encoder system: one encoder processes document embeddings (linear encoder), while the second refines label embeddings (GCN

encoder). I use a single GCN layer because the graph is very shallow (maximum depth = 3). Using more layers would likely lead to oversmoothing. Although alternatives such as deeper architectures with skip connections were considered, they did not yield any performance improvement. Thus, this refinement updates each label vector by integrating signals from its direct hierarchical neighbors, allowing the model to represent classes not only through textual similarity but also through their structural relationships in the taxonomy => context enhancer.

N.B : In the adjacency matrix, the self-loop weight is set lower than the parent-child edge weight to favor hierarchical propagation.

7 Experimental results

7.1 Evaluation Method and submissions

To evaluate model performance, I report four complementary metrics: sample-based F1 (F1-sample), micro-averaged F1 (F1-micro), macro-averaged F1 (F1-macro), and the percentage of hierarchy-consistent predictions, defined as the proportion of cases where the predicted probability of a parent class is higher than that of its child class (Note : I could also have implemented a ranking metric like P@1, P@3 of the main paper).

I primarily rely on F1-micro to compare models and select the best-performing configurations. This choice is motivated by the fact that F1-micro reflects global performance across all labels, weighting each prediction equally and thus being more representative in highly imbalanced multi-label settings. In contrast, F1-macro, which gives equal importance to all classes regardless of their frequency, is less informative in our scenario, where rare classes may introduce high variance and unstable estimates. That said, F1-sample could also be used as a selection criterion, as it evaluates performance at the instance level and better captures prediction quality per sample. Our choice of F1-micro is therefore mainly an implementation decision, and I empirically observe that using F1-sample leads to similar conclusions.

Moreover, model comparison is performed on a validation set built from pseudo-labeled data, providing a consistent basis to assess the relative impact of architectural and training choices. For submission, predictions are generated by running the model on the test set, selecting the top 2 or 3 labels per sample based on confidence (choosing only 2 if the third is far below the second).

Finally, regarding training hyperparameters, we aimed to keep them as consistent as possible across experiments. All models are optimized using AdamW, with a learning rate of 5×10 for all parameters, except for the SBERT-based label embedding encoder, for which a smaller learning rate of 5×10 is used to avoid excessive changes to the pretrained representations. We use a batch size of 128, a dropout rate of 0.3, and apply a label masking rate of 0.05 on the target labels during training. Models are trained for 40 epochs, except we used taxonomy loss, where additional epochs are used to better observe performance evolution over successive pseudo-label updates.

7.2 Results summary

Table 1. Silver labels: Top 1 core class (mpnet embeddings (except 1)) + majority vote (28k documents)

Method	F1 micro	F1 sample	F1 macro	% Hierarchy	F1 Kaggle
Linear (BCE + hier loss)	0.48	0.49	0.30	95%	0.52
InnerProduct (BCE + hier loss)	0.57	0.58	0.40	64%	0.41
GCN (BCE + hier loss)	0.54	0.55	0.40	75%	0.42
GCN (BCE + hier loss + hier label embeddings)	0.54	0.55	0.41	77%	0.43
Linear (BCE loss)	0.50	0.51	0.30	81%	0.42
GCN (BCE loss)	0.54	0.55	0.40	70%	0.40
Linear (Taxo loss)	0.49	0.50	0.33	94%	0.55
Linear (Taxo loss) with MiniLM corpus emb	0.47	0.48	0.26	94%	0.53
GCN (Taxo loss)	0.52	0.53	0.41	88%	0.51
Linear (BCE + hier loss) without regularization	0.49	0.50	0.31	94%	0.51
Linear (BCE loss) with noisy corpus emb	0.14	0.15	0.01	80%	0.12
GCN (BCE loss) with noisy corpus emb	0.20	0.20	0.05	71%	0.21

Table 2. Silver labels: Top 1 core class + hierarchy expansion (28k documents)

Method	F1 micro	F1 sample	F1 macro	% Hierarchy	F1 Kaggle
Linear (BCE + hier loss)	0.76	0.76	0.27	97%	0.61
InnerProduct (BCE + hier loss)	0.88	0.88	0.49	80%	0.60
GCN (BCE + hier loss)	0.83	0.83	0.45	91%	0.60
GCN (BCE + hier loss + hier label embeddings)	0.83	0.83	0.45	93%	0.61
Linear (BCE loss)	0.76	0.76	0.28	92%	0.62
GCN (BCE loss)	0.83	0.83	0.46	89%	0.60
Linear (Taxo loss)	0.80	0.80	0.35	94%	0.62
GCN (Taxo loss)	0.83	0.83	0.47	87%	0.60
NoClassifier (only mpnet similarities)	//	//	//	100%	0.51

Table 3. Self-training with "Learnable" Linear Classifier on pseudo-gold labels 9k docs -> +37 percent pseudo labels

Method	F1 micro	F1 sample	F1 macro	% Hierarchy	F1 Kaggle
Self Training (Taxo + consistency loss)	0.96	0.96	0.31	55%	0.59

7.3 Overall performance

Several observations can be made: Hierarchical multi silver labels are more rigid but give better performance, MPNet embeddings outperform MiniLM embeddings across settings, Hierarchical loss improves performance when using top-1 silver labels, especially with hierarchy expansion. Taxo loss generally provides slightly better results than BCE + hierarchical loss (mainly on the mono silver labels. Classic Linear classifiers achieve the highest Kaggle F1 scores compared to GCN, self-training and InnerProduct classifiers. And, hierarchy condition: Maintaining the hierarchy constraint (child probability < parent probability) consistently in most of the time contributes to higher Kaggle F1 scores.

Summary : MPNet embeddings, a multi hierarchical silver labels, a suitable loss function (Taxo or BCE+hier), and a standard Linear classifier are the most effective combination.

By the way, high silver-label-based metrics do not always correlate with Kaggle F1, so hierarchy compliance is crucial for reliable performance evaluation.

N.B : While Kaggle scores are normally not used for comparison, using metrics on silver labels is somewhat "unreliable". For concreteness, Kaggle F1 provides a more tangible reference for performance. By "unreliable", I don't mean completely so, because silver labels are occasionally wrong but not systematically, the noise is roughly uniform, and the hierarchy is generally respected; in this context, absolute F1 scores may not be precise, but the overall trend and ranking between methods are reliable. But for concreteness, Kaggle F1 provides a more tangible reference for performance.

8 Case study and discussion

8.1 Successful example

A clear success is observed with the classic Linear pipeline using MPNet embeddings and silver labels generated from the top-1 core class plus hierarchy expansion (on 28k documents). In this setup, the model achieves: F1 micro = 0.76, F1 sample = 0.76, F1 macro = 0.28, Hierarchy respected = 92 percent, Kaggle F1 = 0.62.

The pipeline works particularly well because it uses three vertical labels in the taxonomy, providing rich hierarchical context. In contrast, using only top-1 silver labels results in a lack of hierarchical information (less good results).

Like I said before, the choice of MPNet embeddings is also crucial. Compared to MiniLM embeddings, MPNet captures semantic similarity more effectively (higher F1 macro, micro, samples), providing a better basis for classification. Even a simple similarity-based approach without a classifier (NoClassifier: Kaggle F1 = 0.51) already encodes much of the hierarchy and meaning, showing that the preprocessing pipeline alone captures a large part of the structure.

Given the already well-structured silver labels, the Linear classifier can effectively separate classes, learning the semantic mapping from MPNet embeddings with ease. By finetuning only the linear head on top of frozen embeddings, it refines the representation, smoothing and densifying it, while being optimized across many labels using a loss suitable for multilabel classification that respects the hierarchy (e.g. TaxoLoss or BCE + hierarchical loss).

Overall, the combination of accurate silver-label generation, hierarchy-aware expansion, robust embeddings, and appropriate loss functions allows a simple Linear classifier to achieve high F1 scores and maintain hierarchical constraints, because this supervision is already strong and structured, making more complex models that add noise or extra layers unnecessary.

N.B : When using multi-silver labels, the hierarchical loss becomes largely redundant because the hierarchy is already embedded in the silver labels; the Linear classifier essentially learns a static hierarchy, which limits adaptability but still yields strong performance.

8.2 Failure examples

This naturally follows the successful example. Indeed, the same pipeline that strongly benefits a simple Linear classifier becomes too rigid for more complex models. In the successful setting, most of the useful information is already encoded in the MPNet embeddings and in the silver labels themselves (including hierarchical structure through label expansion). As a result, adding a GCN on top does not introduce new useful signals but instead perturbs an already well-structured representation.

GCN models achieve higher F1 scores on silver-label-based metrics because they efficiently exploit label correlations and the graph structure already present in the silver annotations. However, this tight alignment with the silver-label distribution also makes them more sensitive to structural noise and annotation bias, leading to over-alignment to silver-label structure. Thus, although GCNs are designed to exploit graph structure, they do not necessarily enforce hierarchical constraints more strictly in this setting. In our case, the hierarchy is not explicitly encoded as a hard constraint in the GCN, but only implicitly through silver-label correlations. Graph propagation therefore mixes information between neighboring labels without guaranteeing that parent labels remain more probable than their children. When silver labels do not explicitly include hierarchical expansion (e.g., only top-1 core class), this propagation can blur hierarchical ordering, leading to violations of the constraint $p(\text{parent}) > p(\text{child})$. In contrast, the Linear classifier preserves the original MPNet semantic space and learns decision boundaries in a more conservative manner, which results in more stable hierarchy compliance and better Kaggle F1 scores.

This behavior is further confirmed when the pipeline is less structured. With noisy or random corpus embeddings, GCNs clearly outperform Linear classifiers, as graph propagation compensates for the lack of semantic structure. In contrast, when embeddings and silver labels are already strong and consistent, the Linear classifier acts as an implicit regularizer: it preserves the semantic space, learns a hierarchy induced by the silver labels (and loss) and generalizes better. This highlights a key limitation of GCNs in this setting: their strength becomes a weakness when the task is already largely solved by preprocessing, making simpler models more effective.

A similar limitation is observed with self-training, which explains why it fails to outperform—and even degrades performance compared to—the supervised Linear baseline. Self-training assumes that model predictions on unlabeled data introduce useful variation and uncertainty that can refine decision boundaries. In our setting, however, the pseudo-labels generated during self-training closely mirror the existing silver-label structure: they are clean and coherent, but exhibit little diversity and almost no uncertainty. As a result, the classifier has little left to discover, and self-training mainly reinforces existing patterns rather than improving them. This effectively introduces noise instead of new information.

This effect is visible in the self-training results: despite extremely high silver-label-based scores (F1 micro/sample 0.96), hierarchical consistency drops sharply (55 percent), and the Kaggle F1 (0.59) does not improve over the best Linear models trained directly on silver labels. This suggests that self-training amplifies confident but locally inconsistent predictions, especially with respect to hierarchy, instead of improving global semantic alignment. In practice, this indicates that self-training would

likely require pseudo-labels with more controlled uncertainty or diversity to be effective; otherwise, it tends to propagate existing biases rather than yielding meaningful performance improvements.

8.3 Model behavior, limitations and possible ameliorations

Overall, like I said before, a key limitation of the proposed approach in this work is that the task becomes too easy once the silver-label generation and embedding pipeline is strong: most of the semantic and hierarchical structure is already encoded before training, leaving limited room for the classifier to learn beyond reproducing this structure.

From a pipeline perspective, one possible improvement would be to rethink how embeddings and silver labels are used during training. As shown in the paper, TaxoClass benefits from uncertainty during both core-class mining and self-training, whereas in our setting the MPNet embeddings and top-1 silver labels are extremely confident. Introducing controlled noise or diversity—such as multiple plausible core classes per document, as implicitly encouraged by the entailment-based scoring in the paper—could create a more meaningful learning signal instead of a near-deterministic mapping.

For the classifier level, the GCN used here remains relatively basic compared to the taxonomy-aware design in the paper. In TaxoClass, the class encoder operates on ego-networks and explicitly aggregates parent and child information in a controlled manner. Possible improvements could include asymmetric propagation, or attention-based variants (e.g., GAT) to better control how hierarchical information flows, rather than uniformly mixing neighbors.

Finally, in our implementation, the self-training procedure remains suboptimal and somewhat unstable. Although a large number of pseudo-labeled samples are added over iterations (as visible in the t-SNE visualization : see Annexe 5), the added data are dominated by a few frequent classes and still contain a non-negligible amount of mislabeling. As a result, the consistency loss has limited impact: instead of clearly refining decision boundaries, self-training mainly reinforces confident but biased predictions. This contrasts with the main paper, where self-training is carefully controlled via confidence-aware target distributions and class-wise normalization, leading to consistent performance gains. Despite these issues, self-training remains a highly promising approach and can clearly lead to significant improvements when properly implemented. In our case, more sophisticated strategies such as class-dependent thresholds, better confidence calibration, or a more faithful implementation of the target distribution would likely be required to fully unlock its potential.

Beyond self-training, several promising improvement directions remain open. These include fine-tuning document and label embeddings, improving graph-based modeling, or exploring stronger text representations such as LLM-based encoders. However, due to time constraints and the cost of extensive experimentation, these directions were not fully explored in this work.

8.4 Personal Note

I invested significant effort over the past month trying various approaches (some of which are not even included in this report), but I could not improve beyond the current pipeline. I feel a bit disappointed because I often felt lost and spent time exploring strategies that did not work, which was somewhat frustrating especially knowing that a simple linear classifier provided our best results. Given the complexity of the task, I admit that I also used LLMs to assist me in certain parts of the project, which helped overcome some difficulties. Nonetheless, I learned a lot from this project across many different aspects, even if it was not always smooth.

9 Annexes

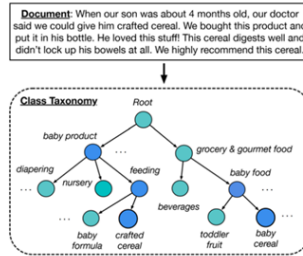


Fig. 1. Annexe 1 : Architecture of the hierarchical classification model

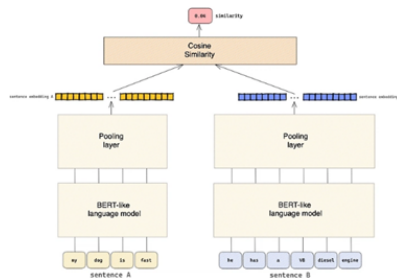


Fig. 2. Annexe 2 : Silver labels generation pipeline

All models 						
Model Name	Performance Sentence Embeddings (14 Datasets) 	Performance Semantic Search (6 Datasets) 	 Avg. Performance 	Speed 	Model Size 	
all-mpnet-base-v2 	69.57	57.02	63.30	2800	420 MB	
multi-qa-mpnet-base-dot-v1 	66.76	57.60	62.18	2800	420 MB	
all-distilroberta-v1 	68.73	50.94	59.84	4000	290 MB	
all-MiniLM-L12-v2 	68.70	50.82	59.76	7500	120 MB	
multi-qa-distilbert-cos-v1 	65.98	52.83	59.41	4000	250 MB	
all-MiniLM-L6-v2 	68.06	49.54	58.80	14200	80 MB	
multi-qa-MiniLM-L6-cos-v1 	64.33	51.83	58.08	14200	80 MB	
paraphrase-multilingual-mpnet-base-v2 	65.83	41.68	53.75	2500	970 MB	

Fig. 3. Annexe 3 : Performances of different sentence transformers

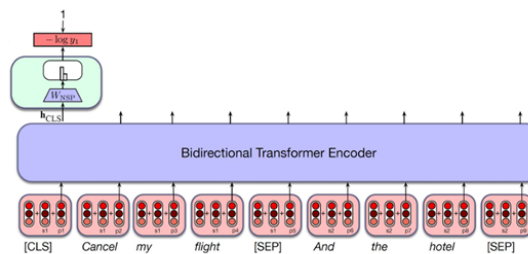


Fig. 4. Annexe 4 : Classification architecture with fixed encoder and linear prediction head

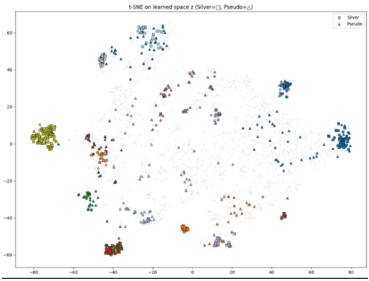


Fig. 5. Annexe 5 : Embedding space learned through self-training (silver labels: squares, pseudo-labels: triangles)